

Software Engineering

1. What is Problem Partitioning in Software Engineering?

Ans: Problem partitioning is the process of dividing a large and complex software problem into smaller and manageable sub-problems.

1. It divides the complete system into smaller modules or components.
2. Each module can be developed and tested independently.
3. It reduces the complexity of the software development process.
4. It makes the system easier to understand, maintain, and modify.

Example: An online shopping system can be divided into modules such as login, product search, payment, and order management.

2. Explain the Importance of Sequence Diagram, Activity Diagram and Class Diagram.

Ans: These are important UML diagrams used to represent different aspects of a software system.

1. **Sequence Diagram:** Shows the sequence of messages or interactions between objects over time.
2. **Activity Diagram:** Shows the workflow or step-by-step activities of a system or process.
3. **Class Diagram:** Shows classes, their attributes, methods, and relationships.
4. These diagrams help developers understand, design, and communicate the system structure and behavior.

3. Differentiate between Top-Down and Bottom-Up Approach.

Top-Down Approach	Bottom-Up Approach
1. Development starts from the main system and moves towards smaller modules.	1. Development starts from small components and combines them to form the complete system.
2. The system is divided into smaller sub-problems.	2. Existing components are combined to create higher-level modules.
3. It focuses on the overall system first.	3. It focuses on individual components first.
4. Commonly used in structured design.	4. Commonly used in object-oriented and component-based development.

4. What is DFD and Context Diagram?

Ans: DFD (Data Flow Diagram) is a graphical representation of how data moves through a software system.

1. It shows data flow, processes, data stores, and external entities.
2. It helps understand how input data is processed and converted into output.
3. A Context Diagram is the highest-level DFD that represents the entire system as one process.
4. It shows the interaction between the system and external entities.

Example: In a banking system, Customer → Banking System → Bank Account Information.

5. Differentiate between Functional and Non-Functional Requirements.

Functional Requirements	Non-Functional Requirements
1. Describe what the system should do.	1. Describe how well the system should work.
2. Define specific functions and services.	2. Define quality attributes and constraints.
3. Example: User can log in to the system.	3. Example: Login should complete within 2 seconds.
4. They are related to system operations.	4. They are related to performance, security, reliability, usability, etc.

6. Explain the Importance of DFD Symbols, Decision Tree and Structured English.

Ans: These are important tools used in structured analysis and system design.

1. **DFD Symbols:** Represent processes, data flow, data stores, and external entities in a clear graphical form.
2. **Decision Tree:** Represents different conditions and their possible decisions or outcomes.
3. **Structured English:** Uses simple English statements with programming logic such as IF, THEN, ELSE, and WHILE.

7. What are Structured Programming/Structured Analysis Tools? What are CASE Tools?

Ans: Structured programming is a programming approach that uses clear control structures such as sequence, selection, and iteration.

1. Common structured analysis tools include DFD, Data Dictionary, Decision Tree, Decision Table, and Structure Chart.
2. These tools help analyze and design complex systems in a systematic way.
3. CASE tools provide software support for activities such as analysis, design, coding, testing, and documentation.
4. CASE tools improve productivity, quality, and consistency in software development.

8. What is a State Chart Diagram? Explain its Role and Importance.

Ans: A State Chart Diagram is a UML diagram that shows the different states of an object and the events that cause it to change from one state to another.

1. It represents the dynamic behavior of a system or object.
2. It shows states, events, transitions, and actions.
3. It is useful for systems where behavior changes according to different events.
4. It helps developers understand and design event-based systems.

Example: An online order can have states: Placed → Confirmed → Shipped → Delivered.

9. What is Object-Oriented Approach?

Ans: The Object-Oriented Approach (OOA) is a software development approach that organizes a system around objects and classes.

1. An object represents a real-world entity such as Student, Employee, or Account.
2. A class defines the properties and behaviors of objects.
3. It uses concepts such as encapsulation, inheritance, polymorphism, and abstraction.
4. It improves reusability, maintainability, and flexibility of software.

10. What is a Class Diagram and Implementation Diagram?

Class Diagram:

Ans: A class diagram is a UML diagram that represents the static structure of a software system.

1. It shows classes, attributes, methods, and relationships.
2. It helps developers understand the structure of the system.
3. It is useful during object-oriented analysis and design.

Implementation Diagram:

Ans: An implementation diagram represents the physical implementation or deployment aspects of a system.

1. It shows software components and/or hardware nodes.
2. It helps understand how software components are organized and deployed.
3. It is useful for planning the actual implementation of the system.

11. What are Frameworks of Environment?

Ans: A framework of environment in software engineering refers to the overall environment that supports software development and operation.

1. It includes tools, processes, methods, standards, and resources required for development.
2. It provides a common structure for activities such as analysis, design, coding, and testing.
3. It helps developers follow a systematic and consistent development process.
4. A proper environment improves productivity, quality, and project management.

12. What is CMMI? Explain its Importance, Benefits, Capability Model and Implementation.

Ans: CMMI (Capability Maturity Model Integration) is a process improvement framework used to improve the quality and maturity of an organization's software development processes.

Importance and benefits:

1. It helps organizations improve and standardize their processes.
2. It reduces errors, risks, and development costs.
3. It improves product quality, productivity, and customer satisfaction.

S.Karak

Capability/Maturity Model:

CMMI uses maturity levels to measure process improvement, commonly from Initial to Optimizing.

Implementation: An organization evaluates its current processes, identifies weaknesses, improves processes, trains employees, and continuously monitors the results.

13. Explain Software Adaptability and Reliability.

Ans: Software Adaptability is the ability of software to adjust to new environments, requirements, technologies, or platforms.

1. Adaptable software can be modified easily when requirements change.
2. It helps software remain useful for a longer period.
3. Example: An application being updated to support a new operating system.

Software Reliability is the ability of software to perform its required functions without failure for a specified period and environment. Reliable software produces correct and consistent results and reduces failures.

14. What is Software Quality Assurance (SQA)?

Ans: Software Quality Assurance (SQA) is a set of planned activities used to ensure that software development follows defined quality standards and processes.

1. It focuses on preventing defects rather than only finding them.
2. It includes activities such as reviews, audits, testing, and process monitoring.
3. It ensures that software meets specified requirements and quality standards.
4. SQA improves the reliability, maintainability, and overall quality of software.

15. What is Formal Technical Review?

Ans: Formal Technical Review (FTR) is a systematic examination of software work products by a group of technical people.

1. It is used to identify errors, defects, and inconsistencies at an early stage.
2. It can be conducted on requirements, design documents, code, or test plans.
3. Reviewers check whether the work follows the required standards and specifications.
4. FTR helps reduce development costs by finding defects before testing or deployment.

16. Differentiate between Software Verification and Validation.

Verification	Validation
1. Checks whether the software is being built correctly.	1. Checks whether the right software is being built.
2. Checks software documents, design, and code against specifications.	2. Checks whether the final software meets user needs and requirements.
3. It mainly involves reviews, inspections, and analysis.	3. It mainly involves testing and actual execution of software.

17. What is Risk Management in Software Engineering?

Ans: Risk management is the process of identifying, analyzing, and controlling possible problems that may affect a software project.

1. Risk Identification: Find possible risks such as cost increase, schedule delay, or technical failure.
2. Risk Analysis: Estimate the probability and impact of each risk.
3. Risk Response: Prepare actions to avoid, reduce, transfer, or accept the risk.
4. Continuous monitoring helps control risks throughout the project.

18. Explain Different Types of Testing.

Ans: Software testing is the process of checking software to find defects and ensure that it meets requirements.

1. **Unit Testing:** Tests individual modules or components separately.
2. **Integration Testing:** Tests whether different modules work correctly together.
3. **System Testing:** Tests the complete software system.
4. **Acceptance Testing:** Checks whether the software satisfies the customer's or user's requirements.

19. What is SDLC?

Ans: SDLC (Software Development Life Cycle) is a systematic process used to develop, test, deploy, and maintain software.

The main stages are:

1. **Requirement Analysis:** Collect and understand user requirements.
2. **Design:** Prepare the architecture, database, interface, and system design.
3. **Implementation/Coding:** Convert the design into program code.
4. **Testing:** Find and fix defects in the software.
5. **Deployment:** Release the software for users.
6. **Maintenance:** Update the software and fix problems after deployment.

1. Breaking a software program into smaller, manageable parts is known as:

- a) Abstraction
- b) Modularity**
- c) Integration
- d) Encapsulation

2. The evolving nature of modern software is mainly due to:

- a) Static requirements
- b) Dynamic requirements**
- c) Fixed architecture
- d) Limited technology

3. Which of the following is NOT a part of a layered technology used in software engineering?

- a) Process
- b) Methods
- c) Tools
- d) Organization structure**

4. CMMI is a framework that provides:

- a) Hardware development processes
- b) Process improvement and maturity guidelines**
- c) Database management
- d) Network security

5. PSP is a software process focused on:

- a) Team-based development
- b) Individual software development**
- c) Hardware development
- d) Database design

6. Software reliability is generally described in terms of:

- a) Probability of failure-free operation**
- b) Number of developers
- c) Program size
- d) Development cost

7. What is the first phase of the Waterfall Model?

- a) Coding
- b) Testing
- c) Requirements Analysis**
- d) Maintenance

8. One major disadvantage of the Waterfall Model is:

- a) Continuous feedback
- b) Backtracking is difficult**
- c) Frequent changes are easy
- d) Early delivery of software

9. Which software development model is best suited for risk analysis?

- a) Waterfall Model
- b) Spiral Model**
- c) V-Model
- d) Prototype Model

10. Which model is useful when the requirements are not clearly understood at the beginning?

- a) Waterfall Model
- b) Prototype Model**
- c) V-Model
- d) Big Bang Model